INSTITUTO FEDERAL
Santa Catarina

Câmpus
São José

# Miniprojeto

Medição ativa com iperf com apoio de medição passiva (tcpdump)

**Curso:** Engenharia de Telecomunicações
**Disciplina:** Avaliação de Desempenho de Sistemas
**Professor:** Eraldo Silveira e Silva

**Aluno(s)**
Wagner Flores dos Santos
Gabriel Kuster Kraus

16 de Março de 2026

# Sumário


**1 Introdução** **2**

**2 Objetivos** **2**
2.1 Objetivo geral . . . . . 2
2.2 Objetivos específicos . . . . . 2

**3 Planejamento experimental formal ($2^2$)** **3**
3.1 Modelo estatístico . . . . . 3

**4 Setup experimental e ferramentas** **3**
4.1 Plataforma computacional . . . . . 3
4.2 Ferramentas utilizadas . . . . . 3
4.3 Topologia emulada . . . . . 4

**5 Procedimento e automação do experimento** **4**
5.1 Aplicação dos fatores . . . . . 4
5.2 Medições . . . . . 5
5.3 Automação: run_experimento.py . . . . . 5
5.4 Análise: analisar_resultados.py . . . . . 6
5.5 Estrutura de diretórios . . . . . 6
5.6 Passo a passo de execução . . . . . 6

**6 Formato e organização dos dados** **6**
6.1 Colunas do arquivo resultados.csv . . . . . 6
6.2 Exemplo de linhas do CSV . . . . . 7

**7 Resultados e discussão** **7**
7.1 Comportamento geral observado . . . . . 7
7.2 Vazão (iperf_avg_mbps) . . . . . 7
7.3 Taxa de retransmissão (retrans_rate) . . . . . 8
7.4 Interpretação física dos resultados . . . . . 9
7.5 Tratamento correto para a métrica de retransmissão . . . . . 10
7.6 $2^2$: efeitos principais e interação . . . . . 10
7.7 Discussão técnica . . . . . 10

**8 Ameaças à validade e limitações** **10**

**9 Conclusão** **11**

**10 Códigos Python e saídas CSV (anexos)** **11**

# 1 Introdução

O protocolo TCP é amplamente utilizado para transporte confiável de dados na Internet. Na prática, o desempenho do TCP não depende apenas da capacidade do enlace, mas também do atraso (*delay*), do algoritmo de controle de congestionamento e da existência de tráfego concorrente no caminho.

Neste mini-projeto foi executado um planejamento fatorial completo $2^2$ com 8 repetições por tratamento, conforme solicitado na especificação do trabalho. O experimento foi implementado em ambiente emulado no IMUNES, utilizando medição ativa com *iperf* e medição passiva com *tcpdump*/*tshark*.

### Fatores (variáveis de entrada)

- **Fator A – Algoritmo TCP:** Reno / Cubic
- **Fator D – Delay no enlace entre roteadores:** 10 ms / 50 ms

### Variáveis de saída

- **iperf_avg_mbps:** vazão média do fluxo TCP principal, obtida via *iperf*;
- **retrans_rate:** taxa de retransmissão TCP, obtida via *tcpdump* + *tshark*.

Além do fluxo TCP principal entre `pc1` e `pc2`, foi adicionado tráfego UDP de fundo entre `pc4` e `pc3`, com o objetivo de aumentar a ocupação do enlace central e induzir condições de congestionamento mais severas. Esse tráfego adicional foi importante para tornar mais visíveis os efeitos do atraso sobre o desempenho do TCP.

O objetivo do experimento foi observar o impacto isolado e combinado do algoritmo de congestionamento (A) e do atraso (D) sobre a vazão e sobre sinais de retransmissão em um cenário com competição por recursos no enlace entre os roteadores.

# 2 Objetivos

## 2.1 Objetivo geral

Avaliar experimentalmente o impacto do algoritmo de controle de congestionamento TCP e do atraso de enlace sobre o desempenho de um fluxo TCP, utilizando medições ativas (*iperf*) e passivas (*tcpdump*/*tshark*) em presença de tráfego de fundo.

## 2.2 Objetivos específicos

1. Implementar um planejamento fatorial $2^2$ com 8 repetições (total de 32 execuções);
2. Medir a vazão média do fluxo TCP via *iperf*;
3. Medir a taxa de retransmissão TCP via análise de PCAP;
4. Inserir tráfego UDP concorrente entre `pc4` e `pc3` para aumentar o congestionamento;
5. Calcular média, desvio padrão e intervalo de confiança de 95%;
6. Estimar os efeitos principais dos fatores A e D;
7. Verificar evidência de interação entre algoritmo e atraso;
8. Interpretar os resultados à luz do comportamento esperado do TCP.

# 3 Planejamento experimental formal ($2^2$)

Foram considerados dois fatores com dois níveis cada, conforme a Tabela 1.

Tabela 1: Fatores e níveis do experimento

| Fator | Nível (-) | Nível (+) |
|---|---|---|
| A – Algoritmo TCP | Reno | Cubic |
| D – Delay | 10 ms | 50 ms |

O número total de combinações é:

$$2^2 = 4$$

Com $r = 8$ repetições por combinação:

$$4 \times 8 = 32 \text{ execuções}$$

## 3.1 Modelo estatístico

O modelo linear adotado para o planejamento $2^2$ foi:

$$y = \mu + A + D + AD + \varepsilon$$

onde $\mu$ é a média geral, $A$ é o efeito principal do algoritmo, $D$ é o efeito principal do delay, $AD$ é o efeito de interação e $\varepsilon$ representa o erro experimental.

O modelo foi aplicado separadamente para:

- vazão média (*iperf_avg_mbps*);
- taxa de retransmissão (*retrans_rate*).

# 4 Setup experimental e ferramentas

## 4.1 Plataforma computacional

Os experimentos foram executados em ambiente local com as seguintes especificações:

Notebook Lenovo ThinkPad Edge E431, processador Intel Core i5, 16 GB de RAM, SSD de 120 GB e sistema operacional Ubuntu.

## 4.2 Ferramentas utilizadas

- **IMUNES** – emulador de redes;
- **Docker** – suporte aos nós virtuais do IMUNES;
- **iperf (2.x)** – geração de tráfego TCP e UDP;
- **tcpdump** – captura passiva de pacotes;
- **tshark** – processamento automatizado de PCAP;
- **Python 3** – automação experimental e análise estatística;
- **tc/netem** – emulação de atraso no enlace.

### 4.3 Topologia emulada

A topologia implementada no IMUNES foi composta por:

- 4 hosts finais: `pc1`, `pc2`, `pc3` e `pc4`;
- 2 roteadores intermediários;
- enlaces Ethernet configurados para 1 Gbit/s;
- delay aplicado apenas no enlace entre roteadores;
- um fluxo TCP principal entre `pc1` e `pc2`;
- um fluxo UDP de fundo entre `pc4` e `pc3`.

O fluxo TCP principal foi usado para medição de vazão e retransmissão. Já o fluxo UDP entre `pc4` e `pc3` foi inserido como tráfego concorrente, atravessando o mesmo enlace entre roteadores e contribuindo para o aumento do congestionamento.

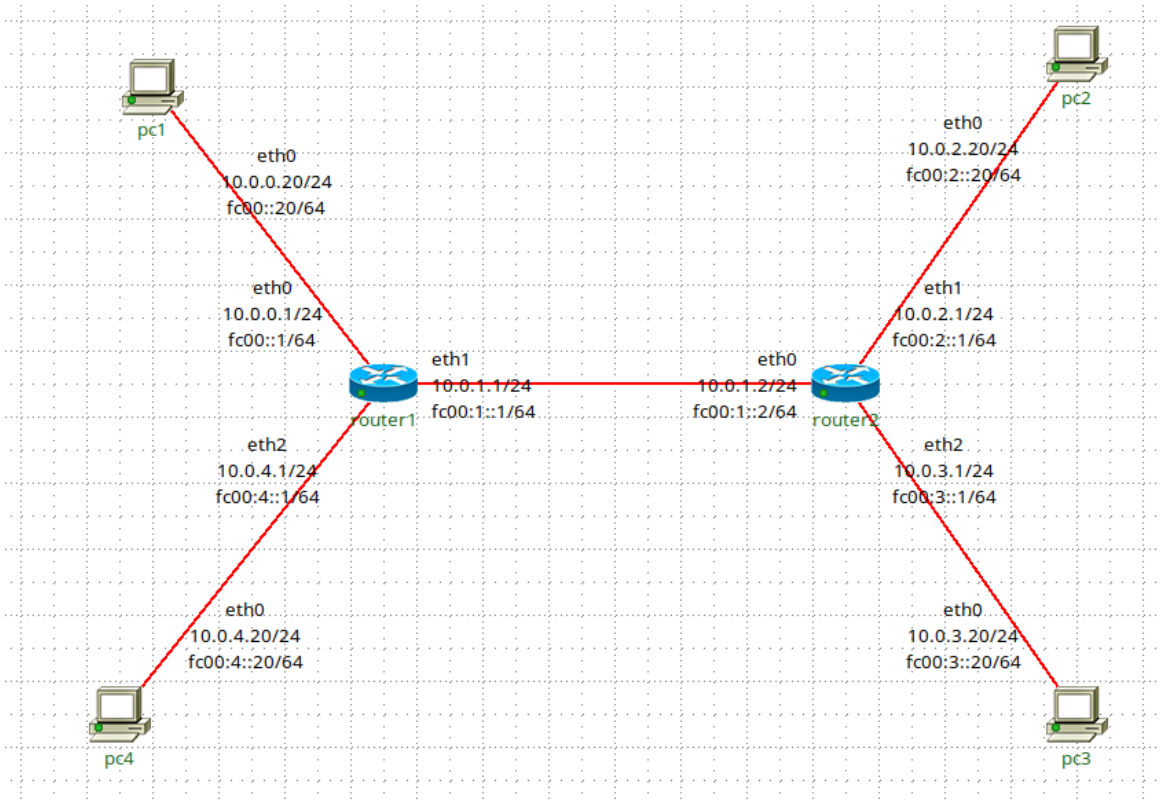


Figura 1: Topologia no IMUNES utilizada no experimento. O fluxo TCP principal ocorre entre `pc1` e `pc2`, enquanto o tráfego UDP de fundo ocorre entre `pc4` e `pc3`.

## 5 Procedimento e automação do experimento

### 5.1 Aplicação dos fatores

Durante as execuções, os fatores foram aplicados diretamente nos nós do IMUNES.

O algoritmo TCP (fator A) foi alterado no `pc1` via:

```
sysctl -w net.ipv4.tcp_congestion_control=reno|cubic
```

O atraso (fator D) foi aplicado no enlace entre roteadores via:

```
tc qdisc add/change dev eth1 root netem delay Xms
```

## 5.2 Medições

### Medição ativa (iperf)

No `pc2` foi iniciado o servidor *iperf* TCP (porta 5001) e no `pc1` foi executado o cliente por 30 s:

```
iperf -c 10.0.2.20 -p 5001 -t 30
```

Ao final de cada execução foi extraído do output do iperf o valor final de taxa, registrado como `iperf_avg_mbps`.

### Tráfego concorrente (UDP)

Além do fluxo TCP principal, foi gerado tráfego UDP de fundo entre `pc4` e `pc3`, utilizando *iperf* em modo UDP:

```
iperf -c 10.0.3.20 -u -p 5002 -b 900M -t 33
```

Esse fluxo foi iniciado alguns instantes antes do TCP principal e mantido ao longo da execução, de modo a compartilhar o enlace entre os roteadores com o fluxo medido.

### Medição passiva (tcpdump + tshark)

A captura foi realizada no enlace entre os roteadores, no `router1`, filtrando apenas o fluxo TCP principal do experimento. Em seguida, o PCAP foi processado com `tshark` no host para obter:

- número total de segmentos TCP com dados (`n_dados`);
- número de retransmissões (`n_retrans`);
- taxa de retransmissão por execução: $R = \frac{n_{retrans}}{n_{dados}}$.

## 5.3 Automação: run_experimento.py

Para viabilizar as 32 execuções com repetibilidade, foi desenvolvido o script `run_experimento.py`. Em cada repetição ele:

1. configura o delay (fator D) no roteador;
2. configura o algoritmo TCP (fator A) no `pc1`;
3. garante o servidor TCP ativo no `pc2`;
4. garante o servidor UDP ativo no `pc3`;
5. inicia o tráfego UDP de fundo no enlace `pc4` $\rightarrow$ `pc3`;
6. inicia o `tcpdump` no enlace entre roteadores;
7. executa o *iperf* TCP no `pc1` por 30 s;
8. encerra a captura de forma controlada;
9. copia o PCAP para o host e processa com `tshark`;
10. registra uma linha no CSV com as métricas.

### 5.4 Análise: analisar_resultados.py

O script `analisar_resultados.py` lê o CSV e calcula:

- médias por tratamento;
- desvio padrão;
- intervalo de confiança de 95% (t de Student);
- efeitos A, D e AD do planejamento $2^2$;
- gráficos de médias por tratamento com IC 95%.

### 5.5 Estrutura de diretórios

```
A1.2/
  run_experimento.py
  analisar_resultados.py
  resultados_imunes/
    resultados.csv
    sumario_iperf_avg_mbps.csv
    sumario_retrans_rate.csv
    sumario_retrans_rate_extendido.csv
    efeitos_2x2.csv
    figuras/
      throughput_means_ci95.png
      retrans_rate_means_ci95.png
```

### 5.6 Passo a passo de execução

1. Abrir a topologia no IMUNES e iniciar a emulação;
2. Garantir que `iperf`, `tcpdump` e `tshark` estejam instalados;
3. Executar o experimento:

   ```
   python3 run_experimento.py
   ```

4. Executar a análise:

   ```
   python3 analisar_resultados.py
   ```

## 6 Formato e organização dos dados

Ao final de cada execução, o script `run_experimento.py` registrou uma linha no arquivo `resultados_imunes/resul`
Cada linha corresponde a uma repetição (`rep`) de um tratamento (definido por `alg` e `delay_ms`).

### 6.1 Colunas do arquivo resultados.csv

- **timestamp**: data/hora da execução;
- **rep**: índice da repetição (1 a 8);
- **alg**: algoritmo TCP (`reno` ou `cubic`);

- **delay_ms**: atraso aplicado (10 ou 50 ms);
- **bg_udp_mbps**: taxa do tráfego UDP de fundo;
- **iperf_avg_mbps**: vazão média do iperf TCP (Mbit/s);
- **tcp_payload_bytes**: total de bytes úteis TCP observados na captura;
- **n_dados**: número de segmentos TCP com dados;
- **n_retrans**: número de retransmissões;
- **retrans_rate**: $R = \frac{n_{retrans}}{n_{dados}}$;
- **pcap_file**: nome do PCAP associado.

### 6.2 Exemplo de linhas do CSV

```
timestamp,rep,alg,delay_ms,bg_udp_mbps,iperf_avg_mbps,tcp_payload_bytes,n_dados,n_retrans,
    retrans_rate,pcap_file
2026-03-16T20:54:37,1,cubic,10,900,523.0,1287059084,23975,3,0.0001251303441084463,cubic_10ms_rep1.
    pcap
2026-03-16T20:18:52,1,reno,10,900,129.0,470326068,25771,495,0.019207636490629003,reno_10ms_rep1.
    pcap
```

## 7 Resultados e discussão

Foram realizadas 32 execuções (planejamento fatorial $2^2$ com $r = 8$ repetições por tratamento), totalizando 4 tratamentos: (Cubic, 10 ms), (Cubic, 50 ms), (Reno, 10 ms) e (Reno, 50 ms).

### 7.1 Comportamento geral observado

Os resultados obtidos apresentaram dois comportamentos bastante distintos:

- Para **delay de 10 ms**, a vazão média foi alta e a taxa de retransmissão permaneceu relativamente baixa;
- Para **delay de 50 ms**, observou-se **colapso severo de desempenho**, com vazão próxima de zero e aumento significativo da taxa de retransmissão.

Esse comportamento ficou ainda mais evidente devido à presença do tráfego UDP concorrente entre `pc4` e `pc3`. Como o fluxo UDP também atravessa o enlace central, ele disputa banda com o fluxo TCP principal e torna o cenário de congestionamento mais agressivo.

### 7.2 Vazão (iperf_avg_mbps)

A Figura 2 mostra as médias por tratamento com IC 95%.

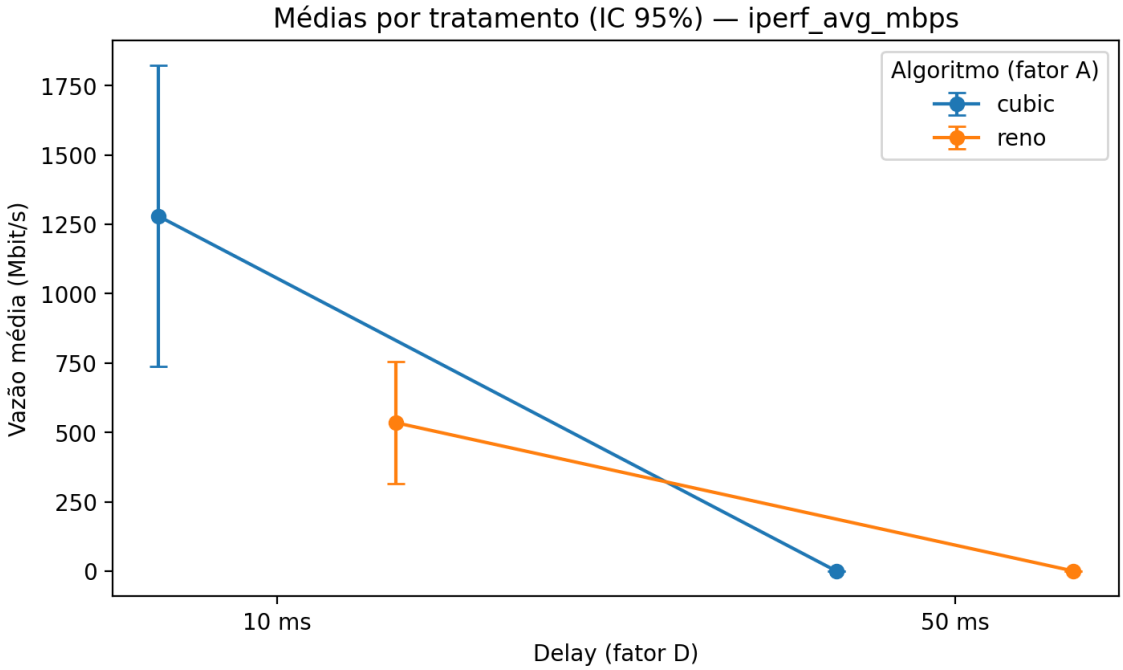


Figura 2: Médias por tratamento (IC 95%) para `iperf_avg_mbps`.

Observa-se que:

- Em **10 ms**, o algoritmo Cubic apresentou vazão média superior ao Reno;
- Em **50 ms**, ambos os algoritmos apresentaram **vazão praticamente nula**;
- A queda de desempenho ao aumentar o delay foi extrema, indicando saturação do sistema.

O comportamento do Cubic em 10 ms sugere maior agressividade no crescimento da janela de congestionamento, o que favoreceu a obtenção de vazões mais altas quando o RTT era baixo. Já em 50 ms, o sistema entrou em regime de forte instabilidade, tornando irrelevante a vantagem do algoritmo.

### 7.3 Taxa de retransmissão (retrans_rate)

A Figura 3 apresenta as médias por tratamento para `retrans_rate`, em escala logarítmica.

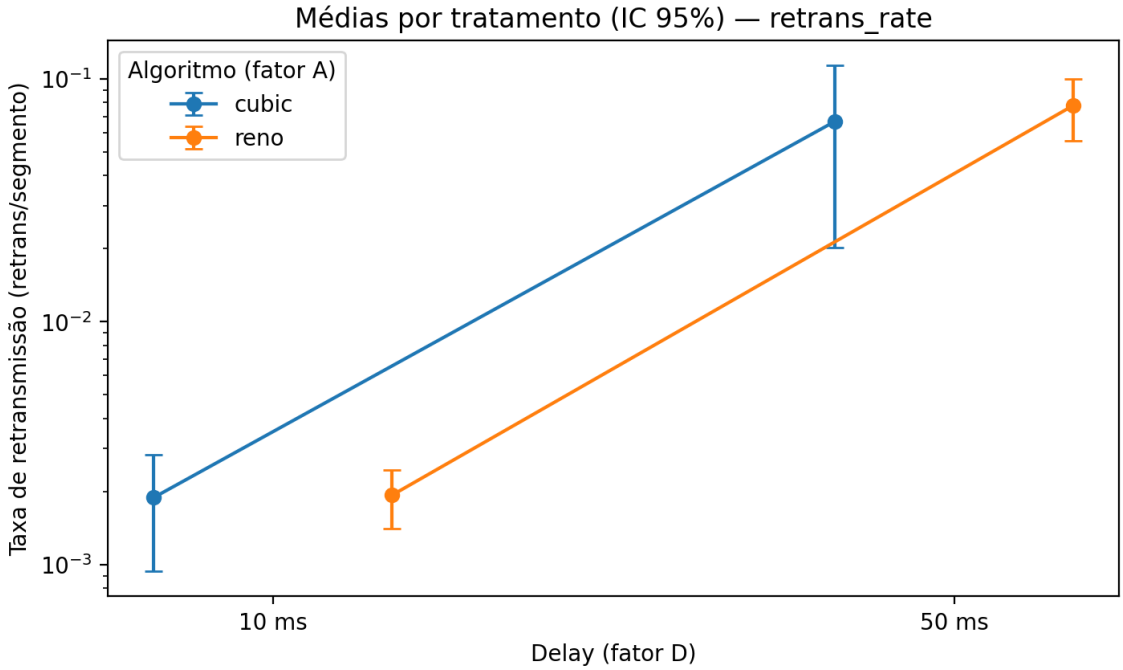


Figura 3: Médias por tratamento (IC 95%) para `retrans_rate` (eixo y em escala log).

Os resultados mostram:

- Em **10 ms**, a taxa de retransmissão permanece baixa, na ordem de $10^{-3}$;
- Em **50 ms**, há aumento expressivo, alcançando a ordem de $10^{-2}$ a $10^{-1}$;
- O Cubic apresentou maior variabilidade em alto delay, enquanto o Reno chegou à maior média observada em 50 ms.

Esse aumento de retransmissões é compatível com um cenário em que o TCP perde pacotes ou detecta congestionamento com maior frequência, reduzindo sua capacidade de sustentar a transmissão útil.

### 7.4 Interpretação física dos resultados

Os resultados sugerem que o **delay (fator D)** foi o principal responsável pela degradação do desempenho.

Quando o RTT aumenta:

- o TCP precisa manter uma janela maior para sustentar a mesma taxa;
- o tráfego UDP concorrente ocupa grande parte da capacidade do enlace;
- o fluxo TCP sofre com perdas, temporizações maiores e retransmissões frequentes;
- a vazão efetiva cai drasticamente.

Em outras palavras, o aumento de delay combinado com carga de fundo elevada entre `pc4` e `pc3` empurrou o sistema para uma condição próxima de **colapso de congestionamento**.

### 7.5 Tratamento correto para a métrica de retransmissão

Como a taxa de retransmissão pode apresentar dispersão alta, especialmente sob congestionamento severo, foi considerado também um conjunto de métricas complementares no arquivo `sumario_retrans_rate_extendido.csv`, incluindo:

- mediana do `retrans_rate`;
- percentual de execuções com retransmissões;
- taxa agregada $R_{agg} = \frac{\sum n_{retrans}}{\sum n_{dados}}$.

Essas métricas ajudam a interpretar os resultados de forma mais robusta, principalmente quando há grande variabilidade entre repetições.

### 7.6 $2^2$: efeitos principais e interação

Do ponto de vista qualitativo, os gráficos indicam:

- **Efeito de D (delay):** extremamente negativo para vazão e positivo para retransmissão;
- **Efeito de A (algoritmo):** perceptível principalmente em baixo delay, com vantagem para o Cubic em vazão;
- **Interação AD:** relevante, pois o impacto do algoritmo depende fortemente do nível de delay.

Essa interação pode ser vista no fato de que, em 10 ms, o Cubic obteve desempenho superior ao Reno, mas em 50 ms ambos convergiram para vazões praticamente nulas.

### 7.7 Discussão técnica

O comportamento observado é coerente com o conceito de produto banda–retardo. À medida que o RTT aumenta, o TCP precisa de mais dados em voo para manter a mesma vazão. Em um cenário com tráfego de fundo intenso e competição pelo mesmo enlace, isso se torna mais difícil, e o protocolo passa a operar de forma menos eficiente.

Assim, o tráfego UDP adicional entre `pc4` e `pc3` teve papel importante no experimento, pois tornou o enlace entre roteadores efetivamente congestionado. Isso permitiu observar mais claramente a sensibilidade do TCP ao atraso e às perdas.

## 8 Ameaças à validade e limitações

**Validade interna.** O experimento depende do correto funcionamento do `tc/netem`, da aplicação do delay na interface correta e da captura no enlace correto entre os roteadores. Um erro de configuração poderia comprometer a validade das métricas.

**Validade de construção.** A métrica `retrans_rate` depende da classificação realizada pelo `tshark`, que utiliza heurísticas para identificar retransmissões TCP.

**Limitações.** O cenário é emulado (IMUNES) e considera apenas dois níveis de delay e dois algoritmos TCP. Além disso, o tráfego concorrente foi mantido fixo em 900 Mbit/s, o que torna o cenário particularmente agressivo.

**Limitação prática observada.** Durante o desenvolvimento da automação, houve necessidade de ajustar a captura de pacotes para evitar geração de arquivos PCAP excessivamente grandes. A versão final do script passou a capturar apenas o fluxo TCP principal, com limpeza automática dos arquivos temporários.

# 9 Conclusão

O planejamento experimental $2^2$ com 8 repetições permitiu avaliar o impacto do algoritmo TCP e do atraso de enlace sobre o desempenho de um fluxo TCP em cenário com congestionamento induzido por tráfego de fundo.

Com base nos resultados obtidos:

- O **delay** foi o fator dominante: ao passar de 10 ms para 50 ms, ocorreu **colapso da vazão**, que se aproximou de zero;
- O **algoritmo Cubic** apresentou melhor desempenho em baixo delay, com vazão média superior ao Reno;
- A **taxa de retransmissão** aumentou significativamente com o delay, alcançando ordens de grandeza muito superiores em 50 ms;
- O **tráfego UDP de fundo entre** `pc4` **e** `pc3` foi decisivo para tornar o enlace central congestionado e evidenciar os efeitos do atraso;
- Houve **interação entre os fatores**, pois a diferença entre os algoritmos dependeu do nível de delay.

De forma geral, os resultados sugerem que, sob alta carga e elevado RTT, o sistema entra em regime de instabilidade, caracterizado por retransmissões frequentes e baixa eficiência de utilização do enlace.

Como continuidade do trabalho, seria interessante repetir o mesmo planejamento com:

- maior duração por execução;
- diferentes níveis de tráfego UDP de fundo;
- inclusão de um fator adicional, como perda ou tamanho de buffer.

# 10 Códigos Python e saídas CSV (anexos)

Repositório do projeto:

`https://github.com/wagnerfloresdossantos/engtelecom-ifsc-sj/tree/main/ads-2026.1/a1.2`[redacted]